

How early can bus bunching be predicted, and is that early enough to act on?

A horizon-resolved machine learning study of eight high-frequency SL bus lines, Stockholm 2024

Bruno Rinaldo and [partner name]

AH2179 Applied AI in Transportation, KTH. Project Option 1 (Public Transport). Code and full analysis: [GitHub repository link]

Abstract. Bus bunching is easy to detect once it has happened and useful to know about only before it does. Using 12.7 million real-time trip-stop observations from eight high-frequency SL bus lines over 2024, joined to weather and service alerts, we predict whether a leader-follower pair will be bunched 1, 2, 3, 5 or 8 stops ahead, and compare a persistence baseline, logistic regression, gradient boosting and a convolutional sequence model on a strictly temporal split. Tuned gradient boosting reaches 0.86 PR-AUC three stops ahead against a 2.4 % base rate, catching 79 % of events at 80 % precision, falling to 34 % eight stops ahead. The usable warning is therefore roughly three to five minutes, which supports a control-room decision aid rather than automated control. Diagnostics show the prediction is carried by delay rather than dwell, contrary to the mechanism we assumed, locate the model's weakness in one specific line rather than in any weather or time-of-day condition, and find no penalty when the model is applied to lines it never saw.

1. Problem and research questions

Bus bunching is a structural instability of high-frequency transit rather than a random failure. A bus that falls slightly behind meets a larger accumulated crowd at the next stop, dwells longer to board it, and falls further behind; the bus behind it reaches stops that were just cleared, dwells less, and closes the gap. Headway variance therefore grows along a route instead of dissipating (Newell and Potts, 1964; Daganzo, 2009). Dwell time, not lateness in itself, is the variable that drives the loop.

The cost lands on passengers as waiting time. For random arrivals at a stop, expected wait is $E[W] = (E[h]/2)(1 + CV^2)$, where CV is the coefficient of variation of the observed headway. Irregularity therefore inflates the average wait even when every scheduled departure runs, which is why a line can be fully delivered and still feel unreliable.

Detecting bunching after the fact is trivial: the headway is already short and any threshold finds it. What an operator can act on is a warning beforehand, because every control action available (holding a bus, short-turning, or simply telling a controller which pair to watch) needs lead time. A one-minute warning and an eight-minute warning are different products, and the difference decides whether the output can drive an automated trigger, support a human decision, or neither.

Research question: how early can bus bunching be predicted from real-time operational data alone, and is that lead time long enough to be operationally actionable? A second question follows from it and decides whether any answer generalises: does a model trained on some lines also work on lines it has never seen? We set no formal hypotheses beyond the expectation built into the first question, that predictive power decays as the horizon grows and stops being useful somewhere; where that point falls is what we measure. Several assumptions we made while building the model turned out to be wrong, and we report those where they arise rather than as separate hypotheses to be adjudicated.

The design choice that shapes the whole study is to treat the prediction horizon as the experimental variable rather than fixing it. Each model is trained and evaluated at five horizons (1, 2, 3, 5 and 8 stops, converted to minutes through the median inter-stop running time of 63 s) and compared against a persistence baseline, which represents what a controller watching a live map already knows without any model.

2. Data and how bunching is measured

Sources. Twelve monthly GTFS-RT files for 2024, about 18.7 million trip-stop observations per month across all modes, joined to the static GTFS stops, routes, trips and transfers and to a service-alert file of 1.97 million rows. Weather came from the Open-Meteo ERA5 reanalysis rather than the source named in the brief, because it is free, gapless for the full year and gridded at a resolution suited to a bus line. Since the reanalysis grid is coarser than the network, stops were snapped to a 0.1 degree grid; nine grid points cover all 380 stops, so nine annual API calls replace thousands of duplicate per-stop requests.

Which lines, and why that choice is a finding. Bunching is only meaningful where headways are short, so candidates were filtered to a median scheduled peak headway of 600 s or less. A completeness scan of the eight highest-volume survivors then showed that real-time coverage is very uneven: two of them, lines 1 and 3, were missing 44 % and 64 % of their observed arrivals, against 1 to 12 % for the rest. Headways computed on those lines would have been mostly interpolation, so they were dropped in favour of the next candidates that still met the frequency bar. The final set is lines 4, 116, 117, 179, 401, 474, 541 and 607: 12.68 million trip-stop rows for the year. All eight are high-frequency trunk services, which bounds what the generalisation test in Section 7 can claim.

Defining the leader. A headway needs a pair of vehicles, and there are two ways to pair them. Sorting arrivals at a stop by observed time reassigns the leader whenever one bus overtakes another, which is precisely when bunching occurs, so any multi-stop trajectory feature built on it would describe a different pair at each stop. We therefore define the leader as the trip scheduled to arrive immediately before, which keeps the pair fixed along the whole trip. The two definitions disagree on 1.4 % of rows, and a quarter of the events our definition calls bunched are invisible to the observed-order one, which is consistent with overtaking being part of the phenomenon rather than noise.

Defining bunching. With $\text{headway_ratio} = \text{observed headway} / \text{scheduled headway}$, a stop is labelled bunched when the ratio is within 0.25 of zero in absolute value. Two properties of that definition matter. It must be absolute, not signed: a follower that has overtaken its leader by 25 minutes produces a large negative ratio, and a signed test would call that bunched when the two vehicles are in fact far apart. It also needs a validity bound, because corrupted pairings produce ratios as extreme as -47 scheduled headways; observed headways are bounded to [-1800, 10800] seconds, which keeps genuine overtaking (93 % of negative values lie within half an hour of zero) and discards the implausible tail. Both points were found by checking the label distribution rather than assuming it, and together they reduced the measured base rate from 2.91 % to 2.29 %, so every number reported here is the conservative version. The label is not knife-edge sensitive: the rate moves from 1.82 % at a 0.20 threshold to 5.35 % at 0.50.

What is missing, and why that is not fatal. About 8 to 10 % of rows have no matched real-time arrival, which propagates to 13.1 % missing headways, and weather is missing for exactly the same 7.9 % of rows whose timestamp is absent, confirming one cause rather than two. Dwell is missing for 16 % of rows, mostly by design, since dwell at a trip's first and last stop is layover rather than boarding. Missingness is also volatile over the year, from under 1 % in good stretches to whole-day outages. Rather than impute, we chose a main model that handles missing values natively and treat feed reliability as a caveat on any seasonal comparison.

3. Descriptive analysis

Bunching affects 2.29 % of valid observations across the eight lines, but the burden is unevenly distributed. Table 1 puts three views of each line side by side: how often it bunches, how irregular its headways are, what that irregularity costs passengers in extra waiting, and how well the model of Section 5 predicts it.

Line	% bunched	Mean headway (s)	CV	Excess wait (min/pax)	PR-AUC, k = 3
4	4.10	573	0.49	1.24	0.846
179	3.06	674	0.58	1.97	0.777
117	2.30	887	0.59	2.63	0.804
607	1.96	861	0.53	2.03	0.877
474	1.88	706	0.52	1.64	0.628
541	1.53	844	0.38	1.04	0.797
401	1.34	914	0.43	1.48	0.809
116	0.87	1061	0.51	2.34	0.922

Table 1. Per-line bunching rate, headway regularity, excess wait caused by irregularity, and per-line model performance (Section 7). Excess wait is observed $E[W] = (E[h]/2)(1 + \text{CV squared})$ minus the $\text{CV} = 0$ timetable benchmark.

Does irregularity grow along the route? On some lines. Lines 4 and 179 show the clean rise the textbook mechanism predicts (line 4 from CV around 0.40 near the start to 0.55 at the end). Lines 116

and 607 are flat, and line 117 steps down after a mid-route gap caused by a short-turn pattern. An apparent drop at line 401's terminus is a sample-size artifact: that stop is observed 1,231 times in the year against roughly 45,000 at earlier stops. Pooling all lines gives a mild upward drift, but the drift is carried by two lines. The growth is therefore real but not a universal law, which is why route position enters the model as a feature rather than as an assumed trend.

Bunching rate by hour of day and line

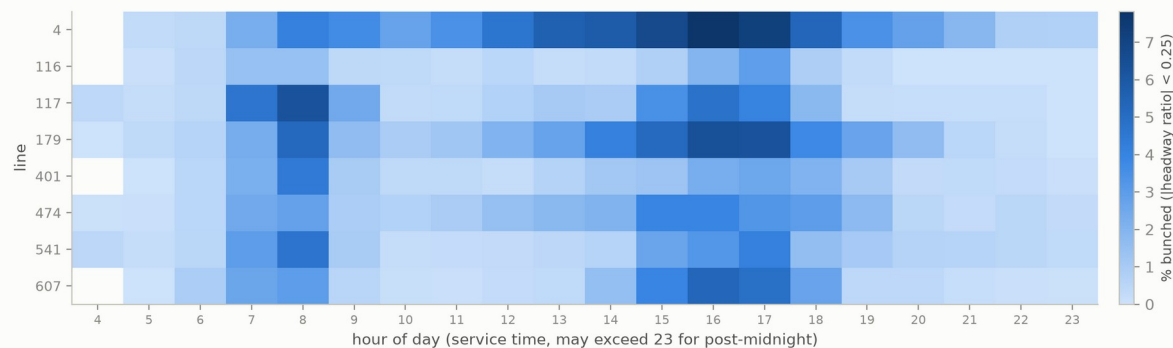
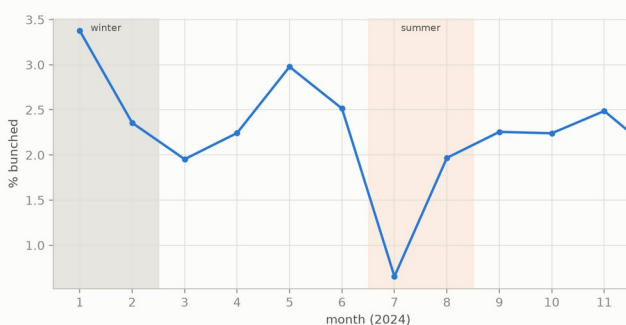


Figure 1. Bunching rate by hour and line. Rush-hour concentration is near-universal, but line 4 is elevated at almost every hour.

When and where. Bunching concentrates in the peaks, with a morning peak on lines 117, 179, 401 and 541 and an afternoon peak on lines 4, 179, 474, 541 and 607 (Figure 1). Line 4 is the outlier of the set, above 7.5 % at 16:00 and elevated at nearly every hour. The spatial view agrees rather than adding evidence: the eight worst stops in the network (Garnisonen 8.04 %, Banérgatan 7.70 %, Skanstull 7.44 % and five others) all sit on line 4's central corridor. For deployment this matters more than the network average, because it says an intervention should be targeted at specific line-hour cells rather than applied as a blanket policy.

Bunching rate by month, all 8 lines pooled



Headway ratio distribution, peak vs off-peak

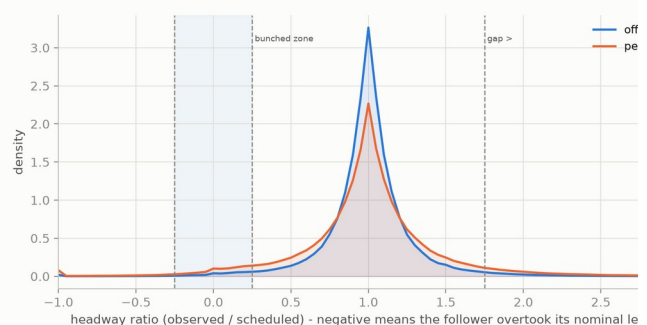


Figure 2 (left). Bunching rate by month: the July low is a timetable effect, not a weather effect. Figure 3 (right). Headway ratio distribution, peak against off-peak: peak service is flatter and fatter-tailed on both sides of the mode.

A seasonal trap worth naming. January is the worst month at 3.38 % and July by far the best at 0.65 % (Figure 2), which invites a weather story. It is mostly the timetable: July has the loosest scheduled headways of the year on these lines (median 900 s against 600 to 650 s from January to May) and among the fewest trips, and a wider scheduled denominator mechanically shrinks a ratio-based bunching rate at unchanged absolute adherence. Combined with the volatile feed coverage noted above, month-level comparison is the weakest evidence in this dataset, so we do not read a weather effect from it and check weather on the model side instead (Section 7), where the confound does not apply.

Peak service is less predictable, not only busier. The peak-hour distribution of the headway ratio is flatter and fatter-tailed than off-peak on both sides of the mode (Figure 3), so peak hours produce more gaps as well as more bunches. The small bump just below zero is genuine overtaking, and it tapers off smoothly well before the validity bound we chose, which is a useful independent check that the bound is not cutting into real data. Practically, a holding policy tuned for the peak has to manage both failure modes rather than only compressing short headways.

The mechanism is cumulative, not instantaneous. Interchange stops have a mean dwell of 39.4 s against 16.3 s at ordinary stops, so the stop attribute tied most closely to boarding load behaves as

the mechanism predicts. But the same-stop correlation between a trip's delay growth and its own headway ratio is only -0.035, and between the leader's delay growth and the follower's ratio only +0.018. Both point the predicted way and neither is strong. We read this not as absence of the mechanism but as evidence that it does not appear at one-stop resolution, which is the direct justification for building multi-stop trajectory features and for testing the dwell mechanism through model ablation rather than through correlation.

4. Modelling approach

Features. 26 numeric, 6 boolean and 3 categorical inputs in eight groups: current state (headway ratio, delays, dwell), trajectory (lagged headway ratios, closing speed over three stops, cumulative delay growth, lagged speeds), the leader's state at the same stop, route position, time context, stop attributes such as interchange status, an active-alert flag, and weather. Weather is exogenous and forecastable, so using it does not compromise a real-time deployment. Temperature enters continuously rather than binned: the expected effect is U-shaped (boarding slows in both cold and heat), but hand-picked bin edges discard information at every boundary, so we give the linear model a quadratic term and let the tree model split the continuous value wherever it likes. No unsupervised clustering was used. The groupings that exist are domain discretisations, and adding a learned stop clustering without evidence that the raw stop attributes were failing would have added complexity we could not justify.

Targets and leakage. For horizon k , the target is whether the same fixed pair is bunched k stops later on the same trip. Trips that end before stop $n+k$ yield a null rather than a zero, and are filtered out before any metric is computed; treating them as negatives would quietly flatter every model. Every feature is computed from stop n or earlier, enforced by construction: feature expressions use only backward-looking windows, and no feature is derived from a forward-looking one. This matters more than it sounds, because a single forward-looking feature would make the whole study meaningless while still producing excellent numbers.

Split. January to September for training (75.3 %), October for validation (8.6 %), November and December for test (16.2 %). A random split is indefensible here: adjacent stops of the same trip are near-duplicates, so a random assignment would put nearly identical rows on both sides of the boundary and inflate the score. A temporal split also stress-tests the model on a season it did not train on. Because both of those are arguments of principle, we tested them: a trip-level random split and a week-based split were trained identically, and on a like-for-like evaluation window the random split gains only 0.007 PR-AUC, with all three landing in a 0.79 to 0.82 band. Leakage from random splitting is real, small, and does not change any conclusion; the temporal split was kept because it is the only one of the three that tests genuine forward prediction. The same comparison exposed a real weakness, discussed in Section 7: October contains no cold weather at all.

Models and metric. Four models of increasing capacity. (1) Persistence, which predicts that the pair's state in k stops equals its state now; this is not a strawman but the standard the model must beat to be worth building. (2) Logistic regression, included for interpretable coefficients rather than accuracy. (3) Gradient boosting (HistGradientBoostingClassifier), chosen over alternatives mainly because it handles the missing headway and weather values natively, without imputing values at exactly the rows where the feed failed. (4) A 1D convolutional network over the six-stop window of headway ratio, dwell and delay growth, with a fourth channel marking which steps were missing, which is also the .keras export the assignment requires. The base rate is 2.0 to 2.4 %, so accuracy is meaningless and PR-AUC is the primary metric, reported with recall at a fixed 80 % precision because that is the number a control room actually needs. Class weights handle the imbalance; the test set's balance was never altered.

5. Results

k	minutes	persistence P / R	LogReg orig / tuned	Keras orig / tuned	GBM orig / tuned	Tuned GBM recall @ 80 % prec.
1	1.05	0.89 / 0.84	0.586 / 0.859	0.917 / 0.932	0.898 / 0.958	0.96
2	2.10	0.84 / 0.75	0.563 / 0.785	0.855 / 0.862	0.860 / 0.905	0.88
3	3.15	0.81 / 0.67	0.539 / 0.721	0.786 / 0.815	0.814 / 0.856	0.79
5	5.25	0.75 / 0.55	0.493 / 0.612	0.682 / 0.709	0.696 / 0.762	0.59
8	8.40	0.67 / 0.41	0.425 / 0.488	0.562 / 0.579	0.623 / 0.652	0.34

Table 2. Test-set PR-AUC (November to December 2024, all eight lines, scored once), before and after tuning. Persistence has no probability score, so its single precision/recall operating point is shown instead. Minutes use the median inter-stop running time of 63 s.

The baseline shows the task is well posed. Persistence achieves 89 % precision and 84 % recall one stop ahead and decays smoothly to 66 % and 41 % at eight stops. Neither a trivially predictable label nor a noise-dominated one would look like that; a steady loss of predictability with distance is the signature of a real process. It also sets a demanding bar at short horizons, where a learned model has to add something beyond knowing whether the pair is bunched already.

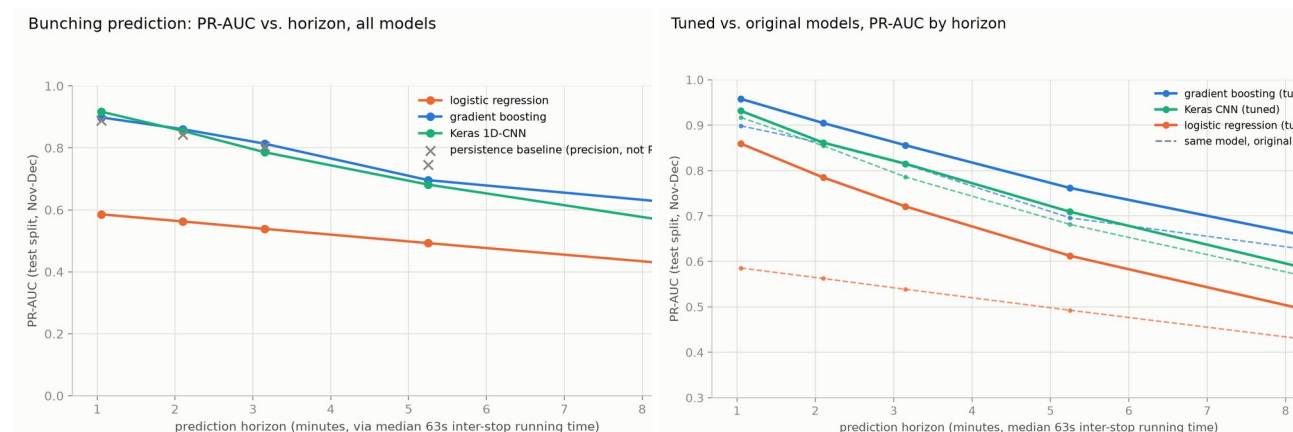


Figure 4 (left). PR-AUC against lead time for all models with the persistence baseline marked as a single operating point. Figure 5 (right). Tuned models against their original configurations; dashed lines are the original settings.

Both nonlinear models clear the baseline, and the margin widens with horizon. Tuned gradient boosting runs from 0.958 PR-AUC one stop ahead to 0.652 at eight, against a 2.4 % base rate, and leads at every horizon. Where persistence loses more than half its recall between the shortest and longest horizon, the learned models keep a usable share of their discriminative power, which is the whole premise for building one: they can see a pair that is closing on its leader but has not bunched yet, and persistence by construction cannot.

What the model relies on. Permutation importance at $k = 3$ is dominated by the current headway ratio (0.73), then arrival delay (0.127), segment speed (0.117) and the leader's arrival delay (0.091). The engineered closing-speed feature, which we expected to dominate, contributes only 0.033. The likely reason is collinearity rather than a useless feature: closing speed is arithmetically a function of the headway ratio and its own lags, which are already in the model, and permutation importance measures what a feature adds given everything else. That is a plausible explanation, not a tested one, and the clean test is listed as future work rather than asserted.

How far ahead the model is usable. The operational answer is in the last column of Table 2 rather than in PR-AUC. Holding precision at 80 %, the model catches 96 % of events one stop ahead, 79 % three stops ahead, 59 % five stops ahead and 34 % at eight. Prediction is therefore strong out to roughly three stops, still useful at five, and misses two thirds of events at eight, which is not a basis for unattended triggering.

6. Tuning, and what it revealed

Tuning was run after the first comparison, on the validation split only. Gradient boosting was searched over twelve configurations (learning rate, leaf count, minimum leaf size, L2, depth) on an 800,000-row subsample and refit on the full training set, gaining 0.03 to 0.07 PR-AUC. The Keras model was searched over nine configurations spanning architecture, pooling, width, dropout, learning rate and batch size, and gained 0.01 to 0.03. Two honest limits: both winners sit at the edge of their search space (largest learning rate and leaf count, deepest and widest network), so more capacity might help further, and both configurations were chosen at $k = 3$ and reused at the other horizons. These are improvements, not optima.

Tuning changed one of our conclusions. Before tuning, the sequence model beat gradient boosting at the shortest horizon and the two looked essentially tied, which we had read as a substantive finding about sequence structure. After tuning, gradient boosting leads at every horizon (Figure 5). The earlier reading was a statement about default settings, not about the models, which is a reminder that a comparison between untuned implementations is a comparison of defaults.

The most useful result came from the weakest model. Logistic regression started far behind at 0.539 PR-AUC at $k = 3$, and we first attributed that gap to the problem being strongly nonlinear. Testing that claim showed it was mostly wrong. Regularisation strength changed nothing (C from 0.01 to 10 moved validation PR-AUC in the fourth decimal, unsurprising with seven million rows and 75 features), and PCA on the original features never helped, only matching the no-PCA score once it retained essentially all the variance, which is what a linear projection should do to a linear classifier: it cannot add capacity, only remove some. The real problem was a representation one. Bunching means the headway ratio is near zero on either side, which is a U-shaped function of a signed variable, and no single linear coefficient can express it. Adding the absolute ratio, its lags and near-zero flags lifted logistic regression from 0.539 to 0.707 test PR-AUC, and PCA on top of those features added a further 0.014, small enough and non-monotone enough that we report it as suggestive rather than as a result. The general lesson is the one worth taking from this project: a linear model that underperforms is often a statement about the features it was given, not about the data.

What tuning does not fix. Gains shrink steadily with horizon, from +0.06 at one stop to +0.03 at eight for gradient boosting. The far horizon is limited by how much signal exists in the present state, not by model settings, and no amount of tuning changes that. Every configuration was also selected on a validation month with no cold weather at all (Section 7), so none was chosen against winter conditions; the test split is the check, and the tuned models hold up on it.

7. Diagnostics and critical discussion

Diagnostics use the original gradient boosting model at $k = 3$. They were not re-run on the tuned model, which is a gap: the conclusions below are qualitative and unlikely to reverse under a moderately better model, but that has not been verified.

Experiment	PR-AUC	Reading
Full model, $k = 3$ (reference)	0.8136	baseline for the rows below
Delay features only	0.8188	best of the three arms
Delay and dwell features	0.8132	adding dwell does not help
Dwell features only	0.8034	weakest of the three arms
Leader features removed	0.7786	graceful, -4.3 % relative
Transfer model, its own 4 lines	0.7696	trained on half the data
Transfer model, 4 unseen lines	0.7726	no transfer penalty (question 2)

Table 3. Ablation, robustness and transfer experiments, each a separately trained model on the same split and horizon.

Delay carries the prediction, not dwell. Delay features alone (0.8188) beat both dwell alone (0.8034) and the two combined (0.8132). Adding dwell on top of delay does not help and mildly hurts. This contradicts the mechanism the project was built on, and it agrees with the independent importance ranking in Section 5, where every delay feature outranks every dwell feature. Two qualifications are needed before calling the mechanism wrong. Dwell is missing for 16 % of rows, so that arm carries more noise. More seriously, the comparison is structurally unfair: the feature set contains a three-stop rolling delay sum but no rolling dwell equivalent, so what was really tested is cumulative delay against single-stop dwell. We therefore report this as delay beating dwell at this feature resolution rather than as a refutation of the dwell mechanism, and building the missing feature is the first item of future work.

The weak spot is a line, not a condition. Per-line performance (Table 1) ranges from 0.922 on line 116 to 0.628 on line 474, a gap far too large for sampling noise at 113,000 test rows, and it is not explained by how much a line bunches, since lines 117 and 179 bunch more and score much better. The false-negative breakdown converges on the same answer independently: line 474 supplies 19.6 % of missed events but only 7.1 % of caught ones. By contrast, performance is flat across peak and off-peak (0.811 against 0.817) although the peak base rate is 3.3 times higher, and flat across weather conditions with adequate sample size (snow 0.814 against clear 0.817). More bunching does not mean more predictable bunching. **This is also where the weather question belongs**, promised in Section 3: which features predict bunching does not change with the weather. Whether adverse weather raises the amount of bunching cannot be established here, because the month-level signal is confounded by the summer timetable and by feed reliability.

How the model fails, and how it survives a degraded feed. Among missed events the mean predicted probability is 0.197, not near zero, so these are borderline cases sitting below the threshold rather than cases the feature set cannot see, and a lower threshold would catch many of them. Removing the leader's features entirely, which simulates one vehicle dropping off the real-time feed and taking with it the leader information of every trip behind it, costs 4.3 % relative PR-AUC. That is a real cost but a graceful one: a deployed system would keep producing weaker predictions through a partial outage instead of going blind.

The second question: transfer to unseen lines. A model trained on four lines scores 0.7726 on the four it has never seen against 0.7696 on its own, a difference within noise. A new line could therefore be onboarded without retraining from scratch. The limits are real: all eight lines are high-frequency trunk routes by construction of the selection rule, so this shows generalisation within that regime rather than to the low-frequency network, and it is a single four-against-four split rather than a leave-one-line-out design with a reported spread.

The validation set is blind to winter. October contains literally zero snowing or ice-risk rows, against 4.2 % in the test months. Every tuning decision, and the sequence model's early stopping, were made against that set, so none was selected with cold conditions in view. The alternative splits we tested fix this but introduce a mirror problem of their own (the week-based split leaves only 0.9 % snow in its test set), so we kept the temporal split and treat this as a stated limitation. The reassurance is empirical rather than structural: performance under snow matches performance under clear skies on the test set.

8. Practical deployment

What the evidence supports is a control-room decision aid at designated control points, not automated network-wide control. A three to five minute warning is enough for an operator to hold a following bus or flag a pair for attention, and the model catches 79 % of events at three stops and 59 % at five while keeping 80 % precision. At eight stops it misses two thirds of events at the same precision, which is not a basis for unattended triggering.

Choosing the threshold is the deployment decision. At the default 0.5 threshold the model catches almost every event but issues more false alarms than true ones, which is a system a control room learns to ignore within a week. At the 80 % precision point, one alert in five is false and roughly three quarters of events are still caught. We report that as a reference rather than a prescription: the right threshold is a judgement about alert fatigue, and it should plausibly differ by line, since per-line performance spans nearly 0.3 PR-AUC. Inference is not a constraint, at 0.006 ms per row when scored in batches, so scoring every active vehicle on each real-time update is free at any credible fleet size.

Where to pilot, and where not to. The central corridor on line 4 is the strongest candidate: the highest bunching rates in the network, high volume, and a line the model predicts comparatively well, so need and confidence coincide. Line 474 should be excluded from automated triggering in a first deployment, since it is the model's worst line both on per-line performance and on its share of missed events.

Benefit, stated as a ceiling. Irregularity alone costs riders on these lines between 1.0 and 2.6 minutes of additional average wait beyond what the timetable implies (Table 1). That is the ceiling if bunching were eliminated entirely; a real holding policy recovers a fraction of it, and we do not estimate that fraction. Two cautions are structural. Holding redistributes waiting time as well as reducing it, and converting a reduction in excess wait into realised passenger-minutes needs a demand model we did not build. More fundamentally, all performance here was measured on an uncontrolled system, while a deployed holding policy would change the dynamics the model learned. A before-and-after pilot at the recommended control points is therefore the only way to validate the benefit.

9. Conclusions, limitations and future work

Answers to the two questions. The first has a concrete answer: bunching can be predicted from operational data alone with about three to five minutes of usable lead time at a defensible false-alarm rate, which makes the output a control-room decision aid rather than an automated controller. Performance decays smoothly with the horizon, and that decay, not the choice of model, is the binding constraint. The second has a positive answer within the regime we studied: a model trained

on four lines predicts four unseen lines with no measurable penalty, so a new line can be onboarded without retraining from scratch, although all eight lines are high-frequency trunk routes and this says nothing about the low-frequency network. Three assumptions we made along the way turned out to be wrong and are worth carrying forward: headway irregularity grows along the route on only two of eight lines rather than universally; delay features predict better than dwell features, contrary to the mechanism the project was built on, although that comparison is structurally unfair to dwell; and logistic regression's weak first showing reflected a missing feature rather than the problem being strongly nonlinear.

Limitations. Eight high-frequency trunk lines out of roughly 536, selected on frequency and data completeness, so neither the descriptive findings nor the transfer result extends to the low-frequency network. Real-time coverage is volatile across the year, which weakens any seasonal reading. The validation month contains no cold weather, and every tuned configuration was selected on it. Searches were small, subsampled and single-seed, with configurations chosen at one horizon and reused at the others, and the diagnostics were computed before tuning. Service alerts enter only as a binary flag, with no text processing, so exogenous disruption is separated from endogenous bunching only crudely.

Future work, in priority order. (1) Add a rolling multi-stop dwell feature and re-run the dwell-against-delay ablation, which is the only way to settle whether the dwell mechanism is genuinely weaker or merely under-represented. (2) Re-run the diagnostics on the tuned model. (3) Repeat the line holdout as leave-one-line-out and report the spread rather than a single number. (4) Investigate line 474 directly, since a deficit of nearly 0.3 PR-AUC concentrated on one line suggests something structural (branching, a terminal pattern, or a shared corridor) that the feature set does not represent. (5) Extend the study to lower-frequency lines, where a longer warning is worth more and the dynamics differ.

References

- Cats, O., Larijani, A.N., Koutsopoulos, H.N. and Burghout, W. (2011). Impacts of holding control strategies on transit performance: a bus simulation model analysis. *Transportation Research Record*, 2216, 51-58.
- Daganzo, C.F. (2009). A headway-based approach to eliminate bus bunching: systematic analysis and comparisons. *Transportation Research Part B*, 43(10), 913-921.
- Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999-2049.
- Moreira-Matias, L., Mendes-Moreira, J., Gama, J. and Ferreira, M. (2015). On improving mass transit operations by using AVL-based systems: a survey. *IEEE Transactions on Intelligent Transportation Systems*, 16(4), 1636-1653.
- Newell, G.F. and Potts, R.B. (1964). Maintaining a bus schedule. *Proceedings of the 2nd Australian Road Research Board Conference*, 2, 388-393.
- Open-Meteo (2024). Historical Weather API, ERA5 archive. <https://open-meteo.com/en/docs/historical-weather-api>
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.